

PROJECT 6

Account Takeover (ATO) Detection Lab

Complete Project Implementation & Learning Reference

This document is a self-contained walkthrough of everything built, tested, broken, and fixed in this project. Read this end-to-end to fully re-learn the project without needing any other source.

Vignesh Gopal Kalaivani

Ontario Tech University — M.Sc. IT Security

1. What This Project Is (and Why It Exists)

This project simulates a real-world Account Takeover (ATO) attack — the scenario where an attacker uses stolen or guessed credentials to log into a legitimate user's account — and builds automated detection for it using a SIEM (Security Information and Event Management) platform.

The project was chosen specifically to target roles in the Canadian financial sector (SOC Analyst L1/L2, MSSP roles at companies like RBC, Scotiabank, eSentire, Difenda). In 2025, the Canadian Anti-Fraud Centre recorded record fraud losses, and incoming Bank Act amendments will legally require Canadian banks to detect and prevent consumer account fraud. This project demonstrates the exact technical skill that requirement demands: correlating multiple weak signals into one strong, actionable alert.

It builds directly on top of a prior project ("Project 5") which already had a working home-lab SOC: a Kali Linux attacker machine, a Debian 13 server running Apache, Wazuh (SIEM), Suricata (IDS), and Cowrie (SSH honeypot), all networked together.

2. Key Concepts to Understand First

2.1 What is Account Takeover (ATO)?

ATO is when an attacker gains unauthorized access to a legitimate user's account, typically using credentials obtained elsewhere (a data breach, phishing, or guessing). The defining detection challenge is that once the attacker has the correct password, their login looks identical to a real user's login — you cannot tell the difference by looking at the successful login alone. You have to look at the behavior around it.

2.2 Credential Stuffing vs. Brute Force (important distinction)

Attack Type	How It Works	Example
Brute Force	One username, many passwords tried against it	Trying 1,000 passwords against 'admin'
Credential Stuffing	Many username/password PAIRS tried, usually from a real data breach dump	Trying leaked Netflix credentials against a bank login, hoping for password reuse

This project specifically simulates credential stuffing, which is more realistic for ATO scenarios — real attackers do not usually brute-force one account for hours; they spray many stolen credential pairs across many accounts.

2.3 What is a SIEM, and What Does Wazuh Do?

A SIEM collects logs from many sources (servers, applications, network devices), normalizes them, and applies rules to detect suspicious patterns. Wazuh is an open-source SIEM made of three components: the Manager (processes logs and applies detection rules), the Indexer (stores and makes alerts searchable, built on OpenSearch), and the Dashboard (the web UI for viewing and investigating alerts).

2.4 Why Correlation Rules Matter More Than Single-Event Rules

A single failed login is meaningless — everyone mistypes a password sometimes. The real signal is behavioral: many failed logins in a short time (volume), followed by a success, from the SAME source. This project's entire design centers on this idea: combining two weak signals into one strong signal is dramatically more accurate and lower-noise than alerting on either alone. This is exactly how real fraud-detection engines and SOC correlation rules are built.

3. Lab Architecture

Kali Linux (192.168.1.10) — the attacker machine, running Hydra to simulate the credential-stuffing attack.

Debian 13 (192.168.1.30) — the target and detection stack, running:

- Apache + PHP — hosting the decoy banking login portal at /securebank/login.php

- Wazuh Manager — evaluates every log line against detection rules
- Wazuh Indexer — stores alerts, makes them searchable
- Wazuh Dashboard — where the analyst investigates alerts and views visualizations

Data flow: Kali sends attack traffic → Apache logs every request/response → Wazuh Manager reads Apache's log file, decodes each line into structured fields, and evaluates custom rules → matching events become alerts → alerts are indexed → the analyst investigates via the Dashboard.

4. Step-by-Step: What Was Actually Built

4.1 Phase 0 — Prerequisites Check

Before building anything, confirmed Apache and Wazuh manager were both running on Debian, and verified Apache was reachable across the network from Kali using curl -I. This confirms the network path an attacker would actually use is functional, not just that services exist locally.

4.2 Phase 1 — Built the Decoy Banking Login Page

Created /var/www/html/securebank/ containing a simple HTML login form (index.html) and a PHP backend (login.php) that checks submitted credentials against two hardcoded fake accounts:

Username	Password
jsmith	Winter2026!
mchen	Toronto123

login.php returns HTTP 200 on a correct login and HTTP 401 on an incorrect one, and logs a custom tagged line via PHP's error_log() function on each attempt. This response-code distinction (200 vs 401) is the exact signal the entire detection system is built to recognize.

Manually tested with curl before any automation touched it, to establish a clean baseline of what a correct login looks like in the logs versus an incorrect one.

4.3 Phase 2 — Built Credential-Stuffing Wordlists

Created two files on Kali: usernames.txt (4 usernames, including the 2 real ones plus 2 decoys) and stuffing_passwords.txt (6 passwords, with the 2 correct passwords deliberately buried mid-list, not tested first). This wordlist design deliberately mimics how a real leaked-credential dump would be used — the attacker does not know in advance which pair is valid.

4.4 Phase 3 — Ran the Credential-Stuffing Attack

```
hydra -L usernames.txt -P stuffing_passwords.txt 192.168.1.30 http-post-form \
"/securebank/login.php:username=^USER^&password=^PASS^:Invalid username"
```

Explanation of the command: -L and -P tell Hydra to try every combination of usernames and passwords (4 x 6 = 24 attempts total). http-post-form is Hydra's module specifically for attacking web login forms via HTTP POST (different from the ssh:// module used to attack SSH in Project 5). The final quoted argument has three parts: the URL to attack, the POST body template with ^USER^/^PASS^ placeholders Hydra substitutes, and the string "Invalid username" which Hydra looks for in the response — if that string is present, Hydra marks the attempt as failed; if absent, it assumes success.

Result: Apache's access log showed a clear pattern — repeated HTTP 401 responses, interspersed with exactly two HTTP 200 responses, both from Kali's IP (192.168.1.10). This confirmed both decoy accounts were successfully compromised by the simulated attack.

4.5 Phase 4 — Ingested Apache Logs into Wazuh

By default, Wazuh in this lab only watched auth.log, syslog, Suricata's alert file, and Cowrie's log (all set up in Project 5) — it had no visibility into Apache's logs at all. Added a new <localfile> entry to Wazuh's main configuration file (/var/ossec/etc/ossec.conf) pointing at Apache's access.log with log_format apache, which tells Wazuh to use its built-in Apache decoder (no custom decoder needed). Verified in the Wazuh Dashboard's

Discover view that Apache events were now flowing in and being parsed into structured fields (source IP, URL, status code).

4.6 Phase 5 — Wrote Custom Wazuh Detection Rules

Wazuh's default Apache rules had no concept that `/securebank/login.php` was a login endpoint — they classified all the attack traffic generically as "Vulnerability Scanning." Five custom rules were written in `/var/ossec/etc/rules/local_rules.xml` to close this gap:

Rule ID	Level	What It Does
100100	0	Base rule — matches any request to the login endpoint. Does not alert by itself; every other rule below builds on top of it.
100101	5	Fires on a single failed login (HTTP 401). Low severity — one wrong password is normal.
100102	10	Fires when 6+ failed logins occur within 60 seconds from the SAME source IP — this is the actual credential-stuffing detector, based on volume and speed, not a single event.
100103	3	Fires on a successful login (HTTP 200).
100104	14	THE CORE RULE. Fires only when rule 100102 (stuffing burst) AND rule 100103 (success) both occur from the SAME source IP within 120 seconds. This correlation of two independent signals is what confirms an actual Account Takeover, not just noisy individual events.

Key Wazuh concepts used in these rules:

- `if_sid` — chains a rule to fire only if another specific rule ID already matched (parent-child relationship)
- `if_matched_sid` — similar, but specifically used for referencing a rule that itself uses frequency/timeframe correlation
- `same_source_ip` — restricts a frequency-based match to events from the identical source IP address, not just any matching events
- `frequency / timeframe` — frequency defines how many matching events are required; timeframe defines the time window (in seconds) they must occur within
- `mitre` — tags a rule with MITRE ATT&CK technique IDs, which Wazuh then displays and allows searching/filtering on

Rules were validated using `wazuh-logtest` — an interactive command-line tool that lets you paste a single raw log line and see exactly which rule(s) match it, without needing to restart the live Wazuh service or generate real traffic. This is the safe way to test rule logic before deploying it.

4.7 Two Real Bugs Encountered and Fixed

BUG 1 — Invalid frequency value. The first version of rule 100104 used `frequency="1"`. Wazuh rejected this with the error: "Invalid frequency: 1. Must be higher than 1 and lower than 10000." Wazuh's correlation engine requires a minimum of 2 matching conditions to correlate against each other — a rule cannot logically "correlate" a single event with itself. Fixed by changing to `frequency="2"`, which is the technically correct value for a rule designed to require two co-occurring signals (the stuffing alert AND the success alert).

BUG 2 — The success rule (100103) never fired against live traffic, despite testing correctly in isolation. Diagnosis: pasting a real HTTP 200 log line into `wazuh-logtest` showed it was being matched by Wazuh's own built-in rule 31108 ("Ignored URLs — simple queries", severity level 0) — NOT rule 31100, which the custom rule 100100/100103 chain depended on via `if_sid`. Root cause: Wazuh's rule engine assigns exactly ONE winning rule to each log line, evaluated like a decision tree. Since 31108 was the rule actually catching this traffic, any custom rule meant to refine detection on 2xx responses had to be chained as a child of 31108 specifically — not assumed based on which parent rule ID seemed logical from documentation. Fixed by changing rule 100103's `if_sid` to 31108, confirmed via `wazuh-logtest` before restarting the live service.

This debugging process is genuinely one of the most valuable parts of the project for interview purposes — it demonstrates real root-cause analysis and Wazuh's actual rule-evaluation model, not just following a tutorial.

4.8 Phase 6 — Triggered and Verified the Detection

Re-ran the Hydra attack after the rule fixes and both bugs resolved. Confirmed via direct inspection of Wazuh's raw alert log (`/var/ossec/logs/alerts/alerts.log`) that the full rule chain fired correctly: rule 100101 fired repeatedly, rule 100102 fired 8 times (confirming multiple stuffing bursts were detected), rule 100103 fired twice (both successful logins), and — the key result — rule 100104 fired twice, once for each compromised account, confirming Account Takeover alerts were generated correctly and in real time.

Verified the same alerts in the Wazuh Dashboard's Discover view by searching rule.id: 100104, confirming full field-level detail: correct source IP attribution (192.168.1.10), correct severity level (14), and correct MITRE ATT&CK tags (T1110 Brute Force, T1078 Valid Accounts).

4.9 Phase 7 — Built Dashboard Visualizations

Built two visualization panels in the Wazuh Dashboard and combined them into a saved dashboard:

- Account Takeover Alerts — a time-series bar chart filtered to rule.id: 100104, confirming a clean spike of exactly 2 alerts at the time of the attack.
- Credential Stuffing Sources — a data table filtered to rule.id: 100102, showing all 88 stuffing-burst detections correctly attributed to the single attacking IP (192.168.1.10), with zero noise from any other source.

Saved as: ATO Detection Dashboard.

4.10 Phase 8 — Wrote the Incident Report

Compiled a full formal incident report covering the executive summary, scenario, architecture diagram, attack execution, detection engineering (including the real bugs and fixes), detection verification with actual alert data, dashboard results, MITRE ATT&CK mapping, recommendations for a real production environment, lessons learned, and a conclusion tying the technical work back to Canadian financial-sector regulatory relevance.

5. Tools Used and What Each One Does

Tool	Purpose in This Project
Kali Linux	Attacker platform
Hydra (http-post-form module)	Simulates the credential-stuffing attack against the web login form
Apache + PHP	Hosts the decoy banking login portal and generates realistic web server logs
Wazuh Manager	Reads logs, decodes them into structured fields, evaluates custom detection rules, generates alerts
Wazuh Indexer	Stores and indexes alerts for search (built on OpenSearch)
Wazuh Dashboard	Web UI for investigating alerts and building visualizations
wazuh-logtest	Command-line tool to safely test rule logic against a single log line before deploying
MITRE ATT&CK framework	Standardized taxonomy for describing attacker techniques — used here to tag T1110 (Brute Force) and T1078 (Valid Accounts)

6. Final Results Summary

- Decoy banking login portal built and functioning correctly (401 on failure, 200 on success)
- Credential-stuffing attack successfully simulated using Hydra, compromising both decoy accounts
- Apache logs successfully ingested into Wazuh with zero custom decoder work needed
- Five custom Wazuh rules authored, debugged, and validated (100100–100104)
- Two real configuration bugs found and fixed through direct system validation (wazuh-logtest), not guesswork

- Two Account Takeover alerts (rule 100104, severity level 14) fired correctly and were verified end-to-end
- Two dashboard visualizations built and saved, confirming clean detection with no false attribution
- Full incident report with architecture diagram produced as a portfolio deliverable

7. How This Fits the Bigger Portfolio

This project builds directly on Project 5's SOC infrastructure (Wazuh + Suricata + Cowrie on Debian, Kali as attacker) and extends it from network/SSH-layer detection into application/web-layer detection — specifically, financial-sector fraud detection. It is the first of a planned sequence of Canadian-financial-sector-focused projects, with natural next steps including a Fraud Pattern Detection rule (impossible travel / velocity checks) and a PCI DSS control-mapping exercise using the same evidence base.